Computer Architecture Scribe

Cache Organization

Anushree C, CO23BTECH11003
Anusha Kumaresan, CS23BTECH11006
Gopu Poojitha CS23BTECH11020
Paidala Vindhya, CS23BTECH11044

17 October 2024, Thursday

Contents

1 Review **1**
1.1 Byte Offset . . . 2
1.2 Examples . . . 2
1.3 Representing sizes of cache . . . 3

2 Spatial Locality **3**

3 Write Access **4**
3.1 Write Through . . . 4
3.2 Write Back . . . 5
3.3 Write Buffer . . . 6

4 Set Associative Cache **6**
4.1 Fully Associative Caches . . . 6
4.2 n-way Set Associative Caches . . . 6
4.3 Analyzing Performance with Associativity . . . 7

---

1 Review

We previously saw that storing data accessed from memory in the cache improves the speed of successive accesses by leveraging the principle of locality. Specifically, we looked at the aspect of locality known as temporal locality, which is based on the idea that memory location that has been accessed recently is likely to be accessed again.
Each location in the cache holds a block of memory, along with tag and valid bit to identify the memory address. In making aligned accesses for words of data from the memory, the block size is taken to be 4. The storage of data as blocks rather than just bytes allows us to ignore the last two bits from the address, referred to as the byte offset.

1.1 Byte Offset

In the direct mapped cache model, memory addresses are mapped to cache indices by extracting tag and index. For example, consider a cache which has 2^{10} blocks and block size of 4 bytes. We are assuming we make aligned accesses to memory. Memory address composed of 64 bits is broken down as

Tag (52 bits)	**Index (10 bits)**	**Byte Offset (2 bits)**

- Since we are making aligned accesses for words, addresses are aligned to multiple of four bytes and the last two (i.e., $log_2 4$) bits which specify the byte location within the word can be ignored in selecting the word from the cache lines. Rightly so, these bits are said to be the byte offset.
- As seen above, we do not include the last 2 bits in finding the index of a memory address in the cache.
- Since there are 2^{10} blocks, we use 10 bits for index and the remaining 52 bits form the tag.
- Instead of accessing words, if we are to access doublewords, which is in multiples of 8 bytes, the last 3 bits of the address would form the byte offset.

1.2 Examples

- **Example 1:**
  Given that: Cache access time = 1 cycle, Memory access time = 100 cycles, Cache miss rate = 10%. What is the average memory access time?
  Solution:
  The Average Memory Access Time can be calculated using the following formula:

  $$\text{Average memory access time} = \text{Cache access time} + \text{Memory access time} \times \text{Miss rate}$$

  Plugging in the values:

  $$\text{Average Memory Access Time} = 1 \text{ cycle} + (100 \text{ cycles} \times 0.1)$$

  Calculating the memory access time contribution from the cache misses:

  $$100 \text{ cycles} \times 0.1 = 10 \text{ cycles}$$

  Now, adding this to the cache access time:

  $$\text{Average Memory Access Time} = 1 \text{ cycle} + 10 \text{ cycles} = 11 \text{ cycles}$$

  What should be the miss rate to have 1.5 cycles as average access time?

  $$\begin{aligned} 1.5 &= 1 + 100 \times M \\ 0.5 &= 100 \times M \\ M &= \frac{0.5}{100} \\ M &= 0.005\,(0.5\%) \end{aligned}$$

  To achieve an access time of 1.5 cycles, the required cache miss rate needs to be 0.5%.

- **Example 2:**
  Given that: Address Size = 32 bits, No. of entries (4 Bytes) in cache = 32. Find the number of bits in each field (tag, index, offset).
  Solution:
  Block size is given as 4B, so byte offset = 2 $[2^2 = 4]$

  Since there are 32 entries in the cache, the number of bits required to index these entries is: $\log_2(\text{number of entries}) = \log_2(32) = 5$

  $$\text{Tag bits} = \text{Total bits} - (\text{Index bits} + \text{Offset bits})$$
  $$\text{Tag bits} = 32 - (5 + 2) = 25 \text{ bits}$$

- **Example 3:**
  Given that: Address Size = 40 bit, No. of entries (8B) in cache = 128. Find the number of bits in each field.
  Solution:
  Block size is given as 8B, so byte offset = 3 $[2^3 = 8]$
  Since there are 128 entries in the cache, the number of bits required to index these entries is:
  $\log_2(\text{number of entries}) = \log_2(128) = 7$

  $$\text{Tag bits} = \text{Total bits} - (\text{Index bits} + \text{Offset bits})$$
  $$\text{Tag bits} = 40 - (7 + 3) = 30 \text{ bits}$$

1.3 Representing sizes of cache

Cache sizes are often represented in powers of 2 because of the binary nature of digital systems. Here's how you can quickly interpret and convert powers of 2 into more familiar units like Kilo, Mega, and Giga. One must be able to quickly identify the decimal equivalent for 2^1 to 2^{10}.

$$2^{10} = 1 \textbf{ Kilo (K)} = 1024$$
$$2^{20} = 1 \textbf{ Mega (M)} = 1024 * 1024$$
$$2^{30} = 1 \textbf{ Giga (G)} = 1024 * 1024 * 1024$$

Example:

$$2^{15} = 2^{10+5} = 2^{10} \times 2^5 = 32 \,\text{KiloBytes (KB)}$$
$$2^{23} = 2^{20+3} = 2^{20} \times 2^3 = 8 \,\text{Megabytes (MB)}$$

- When referring to cache size, it refers to the amount of data from the memory that can be stored in the cache and does not include overheads like tag, valid, dirty bits.
  Eg: What does 16K byte of cache mean? It means total data size is 16K bytes. Suppose we considered block size as
  - 16 bytes, then we have 1K entries
  - 32 bytes, then we have 512 entries

2 Spatial Locality

- So far we have seen how caches leverage temporal locality, i.e., something present in cache is likely to be used again. On the other hand, we also have spatial locality behavior in programs, according to which locations near the addresses accessed are likely to be accessed in the future. Examples with significant spatial locality:

- Fetching instructions sequentially (except in branch/jump instructions).
- Accessing array elements sequentially.

- Cache blocks can be made larger such that they can store data for multiple elements together. Data from memory is fetched in blocks, due to which data from nearby addresses are also fetched.

- Suppose each cache location is of 16 bytes size i.e., each cache block can store 4 words, so accessing the subsequent words in that block would result in hits. By doing this, we get hits for sequential accesses.

- Keeping the index constant, a larger block size results in a smaller tag and a larger offset. On increasing the block size n times, the offset bits are increased by $log_2(n)$ bits while the tag reduces by the same number of bits.

- To write a program where we want to always get a cache miss, we need to know the block size of the cache beforehand and accordingly tune the access stride. Suppose assumed block size as 16 bytes we need to access 0, 16, 32 ... to get misses continuously (note: accessing index 1 after 0 gives a hit due to spatial locality).

3 Write Access

- Accesses from memory can be either for read or write. Reading from cache is already discussed and we saw how it exploits spatial and temporal locality. Now we look at write accesses:

- Why are write accesses treated different for caches?
  - In the case of multi-core systems, after every modification made in that address, we need to update both the cache and the memory.
  - In case of single-core systems, when a different memory address has to be stored in the same index where the previous address got updated in cache, we must ensure that changes made locally in the cache gets saved in the memory before it is replaced with the newer address and its data.

- Maintaining consistency between the cache and memory is a concern when making write accesses. Different write policies can be devised, as described below.

3.1 Write Through

- In the Write-Through policy, the new data value is updated in both the cache and memory at the same time. However, writing to the cache is faster than writing to memory.

- Suppose data got updated at some address, then for this address :
  - if cache hit i.e that address present in cache : Update both memory and cache.
  - else cache miss i.e, that address is not available in cache. Then two choices are :
    - **write allocate** : Fetch that address and its value in cache expecting that if something is written it, we might read it. So will add that address to cache with newly updated value and also update memory.

    * **write without allocate or write no allocate** : Directly updating in memory without changing anything in cache. By doing this no block in cache gets replaced with any new addresses. This policy helps when we just wanted to update some location in memory without reading/writing to it again.

- **Advantages of write-through policy**
  - Consistency in multi core systems - if one core writes a location and another core wants to read it, it can get the latest value from the memory.
  - Avoids overhead of replacement during write-miss i.e., no need to do anything different if that cache block has to be replaced with some newer address and its data.
- **Disadvantages**
  - Since we are always writing to memory, it is slower.
  - In terms of spatial locality, suppose we are updating a few values from the same cache line, then we need to update memory every time for each update. But if we update cache at once and then finally update memory at once, we don't need to access memory many times.
  - Also for i++, don't need to update memory each time it changes i value.

3.2 Write Back

- **BE CAREFUL AND DO NOT CONFUSE THE WRITE BACK CACHE POLICY WITH WRITE BACK STAGE IN PIPELINING**.
- Keep on updating only in the cache unless we need to replace that cache location with some different location. When we need to replace it with a newer address, only then write back the updated cache block to the memory.
- Suppose the address that we are writing back continuously should now be replaced with some newer address. This might delay the access of the new address because firstly we need to write back the existing data to memory.
- But we should write back only the locations which have been modified after reading them from memory. No need of writing back a location if it was not updated and was only read by CPU. How do we know which block data got updated?

Index	**Tag**	**Valid Bit**	**Dirty Bit**	**Data**

  - To distinguish between the blocks we used just to read and blocks whose data got modified, we add another bit called **dirty bit** for each cache block. It tells whether the data in the cache got updated or not after reading from the memory.
  - Whenever we need to replace that block, we check if the block is a dirty block or not. Only if it is a dirty block i.e., if dirty bit is 1, we update the memory and then give space to the newer one.
  - We need this dirty bit only in case of a write back scheme. Not needed in the write through scheme as the memory always has the latest data.
  - Write-back also has the same notions of allocate and without allocate as Write-Through.
- **Advantages**: If repeatedly updating the same block as in spatial locality or in case of i++ or writing back and reading it, happens fast since not updating memory each time the value gets updated.

3.3 Write Buffer

- Whenever there is a write, update the cache and initiate write request to the memory.
- A queue that holds data while the data is waiting to be written in memory is called **write buffer**. But these values have already been updated in the cache.
- When the write to main memory completes, the entry in the write buffer is freed.
- In write buffer model, CPU need not wait until the write to main memory is completed. By this it helps to absorb some of the latency effects.
- If many writes are there, then buffer might get full, then the processor must stall until there is an empty position in the write buffer.

4 Set Associative Cache

- So far, we have seen **direct mapped** cache model where a block of memory maps to exactly one place in the cache. What if we have a program where two arrays are such that their corresponding indices map to the same index of the cache and they are accessed in alternating order? They might keep on removing each other from the cache leading to increased misses. Can we do something to handle this situation?

4.1 Fully Associative Caches

– In a different model, a block of memory can map to any of the locations in cache. In such a **fully associative** cache all the locations must be searched. In the interest of time, such a search for cache hit is performed parallelly with comparators at each cache location. However this significantly increases the hardware cost and proves to be impractical when we have large number of blocks.

– As seen below, we can devise another model known as **set-associative** cache which combines the ideas of direct mapped cache to ease search, and of fully associative cache to reduce the miss rate.

4.2 n-way Set Associative Caches

- Consider the case where we have two memory addresses that map to the same cache line and are to be used alternatively. This results in cache misses since the addresses map to the same location/index and overwrite each other.
- One way to tackle this situation could be to change the mapping such that the addresses in use do not map to the same index in the cache table. Compilers can make such optimizations if it is known that two addresses used in the program map to the same location in cache.
  However, this is not a complete and reliable solution in all cases since two addresses can still collide given any mapping.
- The use of set associative mapping instead of direct mapping can handle collisions to some extent. This cache model, known as **multi-way caches** is able to handle upto n collisions for a n-way cache while requiring additional hardware.

- Per cache location/index, known as a **set**, there are multiple blocks of data, necessitating extra tag bits to select between the n blocks while also reducing the number of index bits by $log_2 n$.
- This also brings along the need for more complex hardware including multiple comparators to check the tags and valid bits for cache hit among the n blocks and a n-to-1 multiplexer to select the data, apart from basic and, or gates as seen below.

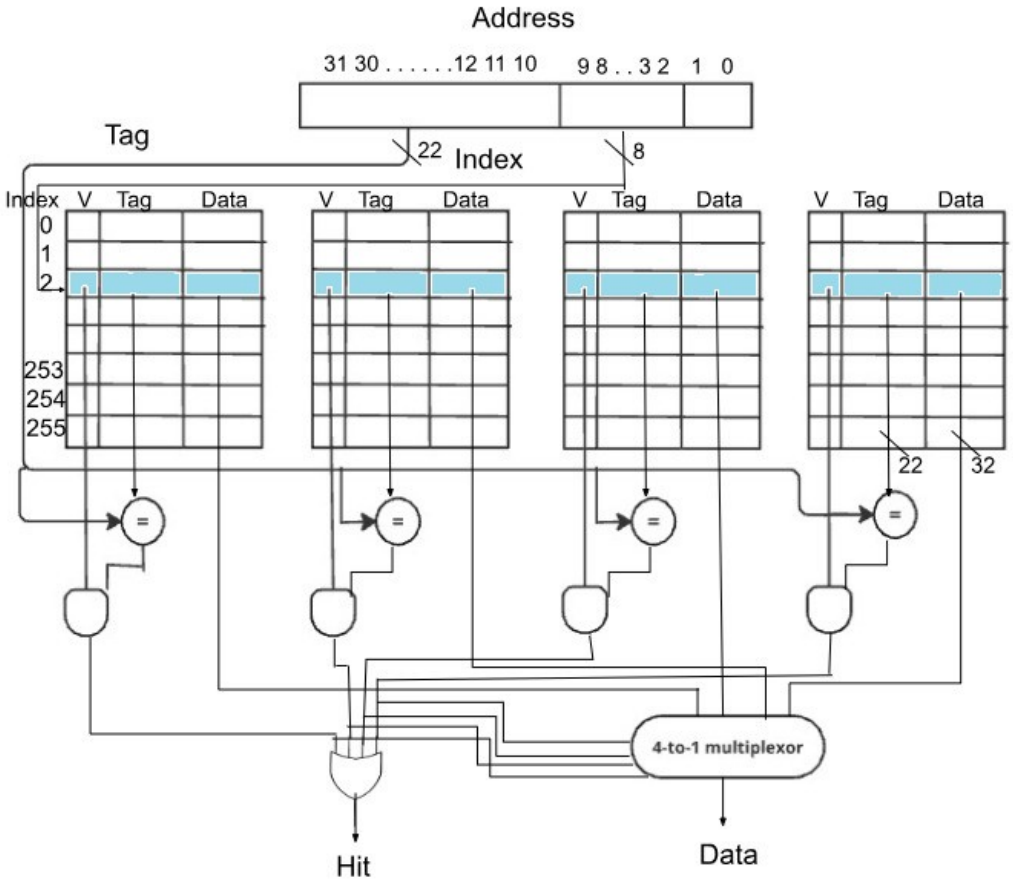


Figure 1: **4-way cache model**

4.3 Analyzing Performance with Associativity

- While associative caches provide the advantage of managing collisions effectively and reducing cache misses in comparison to direct mapped caches, there is a trade-off of the increase in cache access time due to the introduction of more complex hardware components that we must consider.

– In analyzing how much associativity is ideal, we find that increasing associativity reduces miss rate but with diminishing returns. So having very large associativity does not provide much of a decrease in miss rate as opposed to a reasonably smaller number.

– Considering that along with decrease in miss rate, associativity also brings about an undesirable increase in cache access time, the number of ways are chosen. Practical systems too use limited associativity, usually of upto 16.

- The ideas of **A**ssociativity, **B**lock, and **C**apacity are crucial components in the design of caches, and are commonly referred to as the ABC of caches.
- The following Linux command allows to check cache configuration of the system, including cache size, block sizes, associativity:

```
getconf -a | grep CACHE
```